



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

並行程序設計期末實驗報告

SVD 並行化探究

顏國莊

學號：2413140

年級：2024 級

專業：計算機科學與技術

指導教師：王剛

目录

一、 问题背景与描述	1
(一) SVD 分解的核心问题背景	1
(二) 串行 SVD 框架及其并行化挑战	1
(三) 当前并行化解决方案描述	2
二、 问题分析	2
三、 并行算法实现与分析	5
(一) SIMD 与访存优化	5
(二) OpenMP	7
(三) Pthread	9
(四) MPI	11
(五) GPU	13
四、 并行实现拓展与分析	16
(一) 并行方式拓展分析	16
(二) 并行组合拓展分析	19
(三) MPI 进程池改进分析	22
五、 总结	27
(一) 并行方法对比与效果分析	27
(二) 关键优化收获	28
(三) 各次实验收获	29
六、 心得体会	30
七、 大模型使用情况	31
八、 代码链接	31

一、 问题背景与描述

(一) SVD 分解的核心问题背景

SVD 分解的目标是将矩阵 $A \in \mathbb{R}^{m \times n}$ 表示为

$$A = U\Sigma V^T,$$

其中 U 和 V 是正交矩阵, Σ 是非负对角矩阵。若 $m \geq n$, 则 Σ 的前 n 个对角元为奇异值, 通常要求按从大到小排序。SVD 的计算阶段比较复杂: 一方面, Householder 上二对角化包含大量矩阵向量乘和秩一更新, 适合向量化与 GPU; 另一方面, Golub-Kahan 迭代中的 bulge chasing 存在严格顺序依赖, 同一活动块内部不能随意并行。因此, SVD 可以同时检验多种并行思想: 指令级并行、共享内存并行、分布式任务池和异构计算。

(二) 串行 SVD 框架及其并行化挑战

SVD 框架采用两阶段设计。第一阶段是 `to_bidiagonal`, 它通过左右交替的 Householder 变换将一般矩阵化为上二对角矩阵 B , 并同步累积正交矩阵 U 与 V 。第二阶段是 `gkh_svd_from_bidiagonal`, 它在上二对角矩阵上执行 Golub-Kahan 迭代, 利用 Wilkinson 位移和 Givens 旋转追赶 bulge, 使超对角线逐步收敛为 0。最后程序将奇异值调整为非负并降序排列。

串行流程如下:

1. 输入或随机生成矩阵 A ;
2. 调用 `to_bidiagonal` 得到 U, B, V , 保持 $A = UB V^T$;
3. 在 B 上执行 GKH 迭代;
4. 根据超对角线收敛情况不断划分活动块;
5. 对每个活动块执行 `one_block_step`, 完成一次 bulge chasing;
6. 收敛后对奇异值非负化、排序, 并同步调整 U, V ;
7. 输出重构误差、正交误差、对角结构误差和运行时间。

该流程的并行化挑战主要体现在三个方面。

第一, 数据依赖不均匀。Householder 更新中大量元素可以并行更新, 但 GKH 单个活动块内部的 Givens 旋转必须按顺序推进, 后一轮旋转依赖前一轮更新后的矩阵元素。如果直接把 `one_block_step` 内部循环拆给多个线程, 可能破坏 bulge chasing 的数学过程。

第二, 访存模式不均匀。Matrix 类采用行主序存储。对行的连续更新适合缓存和 SIMD; 对列的更新则会产生跨步访问, 难以充分利用 cache line。上二对角化中同一个数学公式, 如果循环顺序不同, 实际性能会明显不同。

第三, 并行开销来源不一致。SIMD 的主要开销是数据对齐和尾部处理; OpenMP/Pthread 的主要开销是线程调度、锁和负载不均; MPI 的主要开销是状态同步、通信和 patch 合并; GPU 的主要开销是 CPU-GPU 数据搬移、kernel 启动和规约效率。

(三) 当前并行化解决方案描述

围绕上述 SVD 框架, 前几次实验已经分别完成了以下工作:

- SIMD 实验: 使用 ARM NEON 指令优化 Householder 与 Givens 中的双精度向量运算;
- 多线程实验: 使用 pthread 与 OpenMP 对 GKH 活动块进行共享内存并行;
- MPI 实验: 实现 rank 0 主进程维护任务池、worker 进程处理子矩阵的分布式版本;
- GPU 实验: 使用 cuBLAS 和手写 CUDA kernel 加速上二对角化阶段。

二、 问题分析

SVD 问题的核心挑战

SVD 并行化并不是对单一循环进行加速, 而是围绕完整分解流程中的不同计算阶段展开。程序首先通过 Householder 变换将原矩阵化为上二对角矩阵, 然后在上二对角矩阵上执行 GKH 迭代, 最后整理奇异值并验证 $U\Sigma V^T$ 的重构误差。因此, SVD 的核心挑战在于: 既要提高矩阵更新和迭代计算的速度, 又不能破坏正交变换累积、奇异值收敛和数值精度。

第一个挑战来自上二对角化阶段。左、右 Householder 变换的核心计算分别可以写为

$$\begin{aligned} w &= v^T B_{sub}, & B_{sub} &= B_{sub} - \beta v w^T, \\ w &= B_{sub} v, & B_{sub} &= B_{sub} - \beta w v^T. \end{aligned}$$

这说明上二对角化主要由 GEMV 和 GER 类操作组成, 其中 GEMV 涉及点积规约, GER 涉及大量矩阵元素更新。当前子矩阵规模越大, 计算量越高, 因此前几轮 Householder 变换往往对总耗时影响最大。本文的 SIMD、OpenMP、GPU 和 cuBLAS 实验都与这一阶段直接相关: SIMD 用于加速点积和连续元素更新,

OpenMP 用于拆分矩阵更新循环, GPU/cuBLAS 则用于检验将 GEMV、GER 交给大规模并行硬件或库函数后的加速效果。

第二个挑战来自 GKH 迭代阶段。GKH 迭代通过左右 Givens 旋转逐步消去上二对角矩阵的超对角线元素。单个活动块内部存在 bulge chasing 过程, 第 i 次旋转会影响第 $i+1$ 次旋转的输入, 因此块内不能简单完全并行。但是, 当某些超对角线元素满足收敛判定后, 矩阵可以被切分为多个独立活动块。本文实验中的 `split_active_blocks` 和 `one_block_step` 正是围绕这一结构展开: 块内保持必要的顺序, 块间通过 OpenMP、Pthread 或 MPI 分配给不同线程或进程。由于活动块长度不同, 工作量可近似按 len^2 估计, 因此负载均衡也是实验分析中的关键问题。

第三个挑战来自完整奇异向量的维护。本文不是只求奇异值, 而是需要得到完整的 U, Σ, V 。因此, Householder 变换和 Givens 旋转不仅要更新中间矩阵 B , 还要同步更新 U 和 V 。这会显著增加实际计算量, 也使访存方式对性能产生影响。连续行更新更适合 SIMD 和缓存优化, 而跨步列更新可能受到内存带宽限制。实验中如果只观察局部循环速度, 而不统计完整 SVD 总时间, 就可能高估某一种并行方法的实际效果。

需要并行化的关键部分

综合以上分析, SVD 中最需要并行化的部分主要包括以下几个方面。

- Householder 变换中的矩阵向量乘法。该部分在上二对角化阶段反复出现, 计算量大且具有规约特征。对每一列或每一行的点积可以使用 SIMD、OpenMP reduction 或 GPU 规约加速。
- Householder 变换中的秩一更新。该部分需要更新当前子矩阵的大量元素, 是典型的数据并行计算, 适合使用 SIMD、OpenMP、GPU kernel 或 cuBLAS GER 操作加速。
- Givens 旋转中的行列线性组合。每次旋转确定 c, s 后, 相关两行或两列的更新可写成线性组合。虽然旋转之间存在顺序关系, 但单次旋转内部的向量更新可以使用 SIMD 提高效率。
- GKH 迭代中的活动块任务。不同活动块之间相互独立, 适合作为 Pthread、OpenMP 和 MPI 的任务划分单位。由于不同活动块长度差异会带来负载不均衡, 任务调度时需要结合 len^2 一类的工作量估计。
- GPU 版本中的 GEMV 和 GER 基本算子。SVD 的上二对角化阶段与 BLAS 操作高度吻合, 因此可以比较手写 GPU kernel 与 cuBLAS 实现的差异, 分析库函数在访存合并、线程组织和规约优化方面的优势。

- MPI 版本中的任务粒度控制。MPI 并行需要在计算量和通信量之间取得平衡。任务过细会导致通信和同步开销过大，任务过粗又可能导致部分进程空闲，因此需要结合活动块长度和阶段耗时分析并行效率。

实现并行化：简单方法与策略

实现 SVD 并行化的核心方法是任务分割与硬件加速。由于 SVD 同时包含稠密矩阵更新、规约计算和块内顺序迭代，不能对所有循环使用同一种并行方式，而应根据不同阶段的计算特征选择合适策略。

- 使用 SIMD 加速局部向量计算：

方法：在 Householder 变换和 Givens 旋转中，存在大量形式相同的浮点运算，例如点积、向量范数、矩阵元素更新以及两行或两列的线性组合。对于

$$B_{ij} = B_{ij} - \beta v_i w_j$$

这类连续数据更新，可以使用 SIMD 指令一次处理多个 double 数据，减少循环次数并提高寄存器利用率。在 Givens 旋转中，当旋转参数 c, s 已经确定后，对 U, V 中相关向量的更新也可以按 SIMD 方式批量执行。

- 使用多线程实现矩阵更新和活动块并行：

方法：在共享内存环境下，可以使用 OpenMP 或 Pthread 对 SVD 中计算量较大的循环进行拆分。上二对角化阶段的 GEMV 和 GER 操作可按行、按列或按矩阵块划分给多个线程；GKH 阶段则不直接拆分单个活动块内部的 bulge chasing，而是将 `split_active_blocks` 得到的多个独立活动块分配给不同线程处理。OpenMP 适合通过 `#pragma omp parallel for` 快速实现循环并行，Pthread 则适合手动维护任务队列，并根据活动块长度 len^2 估计工作量以改善负载均衡。

- 使用 MPI 实现粗粒度任务分发：

方法：MPI 版本采用多进程协作方式，将较大粒度的 SVD 子任务分发给不同进程。主进程可以负责读取矩阵、划分任务和汇总结果，工作进程负责执行局部 Householder 更新或活动块迭代。由于 MPI 通信开销较大，任务划分不能过细，应尽量选择计算量较大的矩阵块或活动块作为分发单位。对于 GKH 迭代，活动块之间相互独立，适合作为 MPI 的基本任务；对于上二对角化阶段，则需要权衡数据广播、结果回收和本地计算量之间的关系。

- 使用 GPU 或 cuBLAS 批量处理核心线性代数操作：

方法：GPU 适合处理上二对角化阶段中规模较大、结构规则的 GEMV 和 GER 操作。可以将矩阵和 Householder 向量复制到显存，使用 CUDA kernel 并行完成矩阵向量乘法和秩一更新；也可以调用 cuBLAS 中的基础线性代数算子，由库函数完成线程组织、访存合并和规约优化。对于小规模矩阵，GPU 启动和数据传输开销可能抵消计算收益；对于较大矩阵，GPU/cuBLAS 更容易体现吞吐量优势。

- 结合 CPU 与 GPU 进行阶段划分：

方法：SVD 中并不是所有部分都适合放到 GPU 上执行。Householder 阶段的密集矩阵更新适合 GPU 或 cuBLAS，而 GKH 阶段的活动块判断、收敛检测和任务调度更适合由 CPU 控制。实际实现时，可以让 CPU 负责算法流程、活动块划分和误差判断，将计算密集的 GEMV、GER 或批量向量更新交给 GPU，从而减少单一设备上的性能瓶颈。

三、 并行算法实现与分析

(一) SIMD 与访存优化

简要介绍

SIMD 的全称是 Single Instruction Multiple Data，即单指令多数据。它的基本思想是在一条指令中同时处理多个数据元素，从而提高重复数值计算的吞吐量。在 SVD 实验中，矩阵元素主要采用 double 类型存储。在 ARM 平台上，NEON 128 位向量寄存器一次可以容纳两个 double，因此非常适合处理矩阵行更新、Householder 秩一更新、Givens 行旋转等具有相同计算形式的循环。

上二对角化阶段的 Householder 变换包含大量矩阵向量乘和矩阵秩一更新，计算形式规整，访存模式相对连续，是 SIMD 优化的重点；GKH 迭代阶段中的左 Givens 旋转需要对矩阵两行做线性组合，也适合向量化；但右 Givens 旋转涉及矩阵两列更新，在行主序存储下访存不连续，SIMD 收益会受到限制。因此，SIMD 在本实验中更适合作为 CPU 端的底层热点优化，而不是单独覆盖整个 SVD 流程的并行方案。

实现思路及方法

在 lab2 中，SIMD 优化主要出现在 bidiagonalization.cpp 和 gkh.cpp 中。由于 Matrix 类采用行主序存储，因此实现时首先需要保证内层循环尽量沿行方向连续访问。只有连续访问时，vld1q_f64 才能直接加载相邻的两个 double，避免额外的数据重排开销。

在上二对角化阶段，左 Householder 变换需要计算

$$w_j = \sum_i v_i B_{ij},$$

随后执行

$$B_{ij} = B_{ij} - \beta v_i w_j.$$

代码中先使用 `vdupq_n_f64` 将标量 v_i 广播到向量寄存器，然后使用 `vld1q_f64` 连续读取两个矩阵元素和两个向量元素，最后用 `vfmaq_f64` 完成融合乘加。

```

1  for (int i = 0; i < rows; ++i) {
2      float64x2_t vi = vdupq_n_f64(v[i]);
3      int j = 0;
4      for (; j + 2 <= cols; j += 2) {
5          float64x2_t bv = vld1q_f64(&B.at(k+i, k+j));
6          float64x2_t wv = vld1q_f64(&w[j]);
7          wv = vfmaq_f64(wv, vi, bv);
8          vst1q_f64(&w[j], wv);
9      }
10     for (; j < cols; ++j) {
11         w[j] += v[i] * B.at(k+i, k+j);
12     }
13 }

```

在矩阵更新阶段，同样沿行方向连续推进。此时先广播 βv_i ，再对两个连续元素同时执行

$$B_{ij} = B_{ij} - \beta v_i w_j.$$

```

1  for (int i = 0; i < rows; ++i) {
2      float64x2_t scale = vdupq_n_f64(beta * v[i]);
3      int j = 0;
4      for (; j + 2 <= cols; j += 2) {
5          double* bptr = &B.at(k+i, k+j);
6          float64x2_t bv = vld1q_f64(bptr);
7          float64x2_t wv = vld1q_f64(&w[j]);
8          bv = vfmsq_f64(bv, scale, wv);
9          vst1q_f64(bptr, bv);
10     }
11     for (; j < cols; ++j) {
12         B.at(k+i, k+j) -= beta * v[i] * w[j];
13     }
14 }

```

在 GKH 阶段，左 Givens 旋转可以写成两行的线性组合：

$$row'_0 = crow_0 + srow_1, \quad row'_1 = -srow_0 + crow_1.$$

由于两行内部按列连续访问，因此可以用 NEON 一次处理两个 double。

```

1 double* row0 = &M.at(r0, 0);
2 double* row1 = &M.at(r1, 0);
3
4 float64x2_t vc = vdupq_n_f64(c);
5 float64x2_t vs = vdupq_n_f64(s);
6 float64x2_t vns = vdupq_n_f64(-s);
7
8 for (; j + 2 <= cols; j += 2) {
9     float64x2_t va = vld1q_f64(row0 + j);
10    float64x2_t vb = vld1q_f64(row1 + j);
11
12    float64x2_t new_a = vfmaq_f64(vmulq_f64(vs, vb), vc, va);
13    float64x2_t new_b = vfmaq_f64(vmulq_f64(vc, vb), vns, va);
14
15    vst1q_f64(row0 + j, new_a);
16    vst1q_f64(row1 + j, new_b);
17 }

```

效果分析

SIMD 优化的关键在于计算形式和访存形式是否同时满足要求。Householder 更新中，子矩阵规模较大，循环体内计算公式固定，并且可以通过调整循环顺序实现连续访存，因此 SIMD 效果较明显。Givens 左旋转同样具有连续行访问特征，也能从 SIMD 中受益。相比之下，Givens 右旋转需要更新两列，在行主序存储中相邻元素之间存在跨步，难以像行更新那样直接连续加载，因此向量化效果较弱。

因此，SIMD 在本实验中的定位是加速 CPU 端局部热点。它能够减少 Householder 和部分 Givens 更新的指令数量，提高浮点运算吞吐量，但不能改变 GKH 单个活动块内部的顺序依赖，也不能替代线程级或 GPU 级并行。

(二) OpenMP

简要介绍

OpenMP 是一种面向共享内存系统的并行程序设计接口，常用于多核 CPU 上的循环并行化。它通过编译指导语句的方式描述并行意图，例如使用 `#pragma omp parallel for` 将循环迭代分配给多个线程执行。OpenMP 的优点是使用简单、代码侵入性低，不需要程序员手动创建线程、销毁线程和管理互斥锁；缺点是对任务队列、线程生命周期和细粒度负载统计的控制能力不如 Pthread。

在 SVD 实验中, OpenMP 适合用来处理 GKH 迭代中不同活动块之间的并行。GKH 单个活动块内部存在 bulge chasing 的顺序依赖, 不能直接把 `one_block_step()` 内部拆成多个线程同时执行; 但是同一轮迭代中, 不同活动块之间由已经收敛的超对角线分隔, 彼此相对独立, 因此可以把不同活动块分配给不同线程。

实现思路及方法

OpenMP 版本仍然沿用“块间并行、块内串行”的思路。每一轮 GKH 迭代开始时, 程序先调用 `split_active_blocks()` 得到当前未收敛的活动块。随后将长度大于 1 的活动块加入任务列表, 已经退化为 1×1 的块不再作为任务处理。之后使用 OpenMP 对任务列表进行并行遍历, 每个线程领取一个 block, 并调用 `one_block_step()` 完成该活动块的一次迭代。

```
1 std::vector<Block> tasks;  
2 for (int i = static_cast<int>(blocks.size()) - 1; i >= 0; --i)  
3 {  
4     if (blocks[i].r > blocks[i].l) {  
5         tasks.push_back(blocks[i]);  
6     }  
7 }  
8 #pragma omp parallel for num_threads(SVD_THREAD_NUM) schedule(  
9     dynamic, 1)  
10 for (int i = 0; i < static_cast<int>(tasks.size()); ++i) {  
11     Block blk = tasks[i];  
12     one_block_step(U, B, V, blk.l, blk.r);  
13 }
```

这里选择 `schedule(dynamic, 1)`, 是因为不同活动块长度不同, 计算量差异较大。若采用静态调度, 线程虽然可能分到数量相近的任务, 但长块和短块的实际计算量并不相同, 容易出现部分线程提前结束而等待长块线程的情况。动态调度允许线程完成当前任务后继续领取下一个任务, 能够缓解活动块长度不均带来的负载不均衡。

此外, OpenMP 版本没有在 `one_block_step()` 内部使用 `critical` 或显式互斥锁。原因是不同活动块作用于不同的子区间, 块间由收敛的超对角线分隔; 只要任务划分正确, 同一轮中的不同线程不会同时修改同一个活动块。这样可以避免把并行区域重新串行化。

效果分析

OpenMP 版本的优势是实现简单, 能够快速验证 GKH 活动块之间是否存在可利用的并行度。当一轮迭代中存在多个非平凡活动块时, OpenMP 可以把这些

活动块分配给不同线程，从而减少该轮的等待时间。但它的加速效果受活动块数量和活动块长度影响很大。若当前轮只有一个大活动块，即使设置 4 个或 8 个线程，也只有一个线程能够执行 `one_block_step()`，其他线程没有可处理任务；若活动块很多但每个块都很小，则 OpenMP 线程调度和隐式同步开销可能抵消并行收益。

因此，OpenMP 在本实验中更适合作为共享内存块间并行的直接实现和对照版本。它能够用较少代码实现动态调度，但对线程生命周期和累计负载统计控制较弱。后续 Pthread 线程池版本正是在这一基础上进一步控制任务队列和 worker 复用，从而减少反复进入并行区域或重复创建线程带来的开销。

(三) Pthread

简要介绍

Pthread 是 POSIX 标准提供的底层线程库。与 OpenMP 相比，Pthread 不依赖编译指导语句，而是由程序员显式创建线程、传递参数、设置互斥锁和条件变量，并在任务结束后进行同步。Pthread 的代码复杂度更高，但控制能力更强，适合实现自定义线程池、任务队列和负载统计。

在 SVD 实验中，Pthread 主要用于 GKH 阶段的活动块调度。由于活动块之间可以并行，而活动块内部需要保持 bulge chasing 的顺序，因此线程的任务单位应当是 block，而不是单个矩阵元素或单次 Givens 旋转。

实现思路及方法

lab3 中实现了非线程池版本和线程池版本。非线程池版本采用较直接的方式：每一轮 GKH 迭代得到当前活动块后，主线程创建固定数量的 pthread 工作线程，每个线程根据编号处理一部分 block，最后主线程通过 `pthread_join()` 等待所有线程结束。这种方式结构简单，但每一轮都要创建和回收线程，开销会随着迭代轮数不断累积。

线程池版本将线程创建从迭代循环中移出。程序开始时创建固定数量 worker，worker 进入等待状态；每一轮主线程生成任务列表后，将任务放入共享队列，通过条件变量唤醒工作线程。工作线程动态领取任务，执行 `one_block_step()`，并记录自己处理的任务数量和估计工作量。所有任务完成后，主线程进入下一轮 GKH 迭代。

```
1 struct Pool {  
2     Matrix *U;  
3     Matrix *B;  
4     Matrix *V;  
5     std::vector<Block> tasks;  
6 }
```

```
7   int next_task;
8   int active_workers;
9   bool has_work;
10  bool stop;
11
12  pthread_mutex_t mutex;
13  pthread_cond_t cond_start;
14  pthread_cond_t cond_done;
15
16  std::vector<pthread_t> threads;
17  std::vector<long long> total_tasks;
18  std::vector<long long> total_work;
19  };
```

工作线程的核心逻辑如下。线程首先等待主线程提交任务；被唤醒后，从任务队列中领取一个活动块；随后释放锁并执行计算；计算结束后重新加锁，更新负载统计信息。

```
1  while (true) {
2      pthread_mutex_lock(&mutex);
3
4      while (!has_work && !stop) {
5          pthread_cond_wait(&cond_start, &mutex);
6      }
7
8      if (stop) {
9          pthread_mutex_unlock(&mutex);
10         break;
11     }
12
13     int id = next_task++;
14     active_workers++;
15     Block blk = tasks[id];
16
17     pthread_mutex_unlock(&mutex);
18
19     one_block_step(*U, *B, *V, blk.l, blk.r);
20
21     pthread_mutex_lock(&mutex);
22     total_tasks[tid]++;
```

```

23     total_work[tid] += block_work(blk);
24     active_workers--;
25
26     if (next_task >= static_cast<int>(tasks.size()) &&
27         active_workers == 0) {
28         has_work = false;
29         pthread_cond_signal(&cond_done);
30     }
31
32     pthread_mutex_unlock(&mutex);
33 }

```

为了更合理地衡量负载, 代码中没有只统计每个线程处理了多少个 block, 而是使用活动块长度估计工作量。若活动块为 $[l, r]$, 长度为

$$len = r - l + 1,$$

则用

$$W = len^2$$

近似该活动块的工作量。这是因为 GKH 中长块的计算量明显大于短块, 只比较任务个数会低估长块造成的负载不均。

效果分析

Pthread 线程池的主要优势是线程复用和动态任务领取。与非线程池版本相比, 线程池避免了每一轮反复调用 `pthread_create()` 和 `pthread_join()` 的开销; 与 OpenMP 版本相比, 线程池可以更直接地保存每个 worker 的任务数、工作量和等待情况, 便于分析负载均衡。

但 Pthread 线程池也不能突破 GKH 的算法依赖。若某一轮只有一个很长的活动块, 则只有一个线程能执行该块, 其余线程仍然需要等待; 若活动块数量很多但都很短, 则互斥锁、条件变量和任务队列操作会成为额外开销。因此, Pthread 的加速效果取决于活动块数量、活动块长度分布和任务粒度。它适合作为 GKH 阶段的共享内存并行实现, 但不能替代上二对角化阶段的 SIMD 或 GPU 加速。

(四) MPI

简要介绍

MPI 的全称是 Message Passing Interface, 即消息传递接口。它主要用于分布式内存系统, 不同进程拥有各自独立的地址空间, 需要通过发送和接收消息完成数据交换。与 OpenMP 和 Pthread 不同, MPI 不依赖共享内存, 因此更适合跨节点或多进程并行。

在 SVD 实验中, MPI 的难点比普通独立任务更高。GKH 活动块虽然可以作为任务分发给 worker, 但每个 worker 在计算时需要访问当前的 B, U, V 状态, 计算结束后还会修改这些矩阵的局部区域。因此, MPI 版本不仅要分发任务, 还要维护矩阵状态同步和结果 patch 合并。

实现思路及方法

lab4 中采用主从式任务池结构。rank 0 作为 master, 负责维护全局矩阵状态、活动块任务池和结果合并; 其他 rank 作为 worker, 负责接收活动块任务并在本地执行 GKH 子问题。

程序开始时先初始化 MPI 环境, 获取当前进程编号和进程总数:

```
1 MPI_Init(&argc, &argv);
2 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
3 MPI_Comm_size(MPI_COMM_WORLD, &size);
```

master 进程根据当前超对角线划分活动块, 并将任务分发给空闲 worker。worker 收到任务后, 先接收 master 发送的最新矩阵状态, 随后在本地执行 process_block_until_resplit()。当活动块再次分裂或达到步数预算后, worker 将局部更新结果返回 master。

```
1 if (rank == 0) {
2     while (active_workers > 0) {
3         MPI_Probe(MPI_ANY_SOURCE, TAG_RESULT_HEADER,
4                   MPI_COMM_WORLD, &status);
5
6         int source = status.MPI_SOURCE;
7         recv_result_patch(source, elapsed, steps,
8                           new_blocks, U, B, V);
9
10        for (Block b : new_blocks) {
11            if (b.r > b.l) {
12                task_pool.push_back(b);
13            }
14        }
15
16        dispatch_one(source);
17    }
18 } else {
19     MPI_Recv(msg, 3, MPI_INT, 0, MPI_ANY_TAG,
20             MPI_COMM_WORLD, &status);
21 }
```



```
22     recv_full_state_from_master(U, B, V);
23
24     std::vector<Block> new_blocks =
25         process_block_until_resplit(U, B, V, blk,
26                                     tol, max_steps, steps);
27
28     send_result_patch(0, blk, elapsed, steps,
29                      new_blocks, U, B, V);
30 }
```

结果回传时, worker 不需要把整个矩阵全部传回, 而是回传被修改的局部 patch, 包括相关的 U 列块、 B 子块、 V 列块以及新产生的活动块。master 收到 patch 后合并到全局矩阵, 并继续分发新的任务。

效果分析

MPI 版本的核心问题是通信开销与计算收益之间的平衡。对于 SVD 来说, worker 在处理活动块之前必须获得足够新的矩阵状态, 否则可能基于过期数据计算; worker 处理结束后, master 又必须正确合并局部更新。因此, 如果每次任务都同步完整 B, U, V , 通信量会随矩阵规模平方级增长, 很容易抵消多进程计算带来的收益。

因此, MPI 更适合粗粒度任务。当活动块较大、worker 能够在本地连续执行较多 GKH 步骤时, 计算时间才可能覆盖状态同步和 patch 回传的开销。若活动块很小, 或者每处理一次都需要完整同步矩阵, 那么增加进程数不一定带来加速, 反而可能因为通信、等待和合并开销导致性能下降。后续优化方向应当是减少完整状态同步, 采用增量同步、非阻塞通信, 或者使用 MPI 负责节点间粗粒度任务分发, 节点内再使用 Pthread 线程池处理局部活动块。

(五) GPU

简要介绍

GPU 具有大量计算核心, 适合执行规则、密集、数据量大的并行计算。CUDA 是 NVIDIA 提供的通用并行计算平台, 允许程序员使用 C/C++ 风格的 kernel 将大规模数据并行任务映射到 GPU 线程上执行。对于 SVD 分解而言, 上二对角化阶段中的 Householder 变换可以分解为 GEMV 和 GER 等规则矩阵运算, 因此适合交给 GPU 加速; 而 GKH 阶段的单个活动块内部存在严格顺序依赖, 不适合直接整体搬到 GPU 上并行执行。

实现思路及方法

lab5 中实现了两类 GPU 上二对角化方案: 一种是基于 cuBLAS 的库函数版本, 另一种是手写 CUDA kernel 版本。cuBLAS 版本利用成熟 BLAS 库完成

矩阵向量乘和秩一更新；手写 kernel 版本则显式实现每个 Householder 更新中的 GEMV 和 GER。

手写 kernel 中，左 Householder 的矩阵向量乘可以表示为

$$w[col] = \sum_i v[i] \cdot B[k+i, k+col].$$

对应 kernel 中，一个 block 负责一个输出列，block 内线程并行累加，并使用 shared memory 进行规约：

```

1  __global__ void left_gemv_B_kernel(
2      const double *B, const double *v, double *w,
3      int m, int n, int k) {
4
5      extern __shared__ double sdata[];
6      int col = blockIdx.x;
7      int tid = threadIdx.x;
8      int rows = m - k;
9
10     double sum = 0.0;
11     for (int i = tid; i < rows; i += blockDim.x) {
12         sum += v[i] * B[(k+i) * n + (k+col)];
13     }
14
15     sdata[tid] = sum;
16     __syncthreads();
17
18     for (int stride = blockDim.x / 2; stride > 0; stride >>= 1)
19     {
20         if (tid < stride) {
21             sdata[tid] += sdata[tid + stride];
22         }
23         __syncthreads();
24     }
25
26     if (tid == 0) {
27         w[col] = sdata[0];
28     }
}

```

随后通过 GER kernel 完成矩阵更新：

$$B[k + row, k + col] = B[k + row, k + col] - \beta v[row]w[col].$$

```

1  __global__ void left_ger_B_kernel(
2      double *B, const double *v, const double *w,
3      double beta, int m, int n, int k) {
4
5      int col = blockDim.x * blockIdx.x + threadIdx.x;
6      int row = blockDim.y * blockIdx.y + threadIdx.y;
7
8      int rows = m - k;
9      int cols = n - k;
10
11     if (row < rows && col < cols) {
12         B[(k+row) * n + (k+col)] -= beta * v[row] * w[col];
13     }
14 }

```

GPU 版本还需要维护主机端和设备端之间的数据流。程序首先在 GPU 上为 B, U, V 和临时向量分配显存, 将输入矩阵从 CPU 拷贝到 GPU; 每轮 Householder 中, CPU 构造当前 Householder 向量并上传到 GPU, 然后调用 GEMV 和 GER kernel 或 cuBLAS 函数完成矩阵更新; 上二对角化结束后, 再将 B, U, V 拷回 CPU, 交给后续 GKH 阶段继续处理。

```

1  cudaMalloc(&dB, bytesB);
2  cudaMalloc(&dU, bytesU);
3  cudaMalloc(&dV, bytesV);
4  cudaMalloc(&dv, maxLen * sizeof(double));
5  cudaMalloc(&dw, maxLen * sizeof(double));
6
7  cudaMemcpy(dB, hA.data(), bytesB, cudaMemcpyHostToDevice);
8
9  left_gemv_B_kernel<<<cols, tB, tB * sizeof(double)>>>
10     (dB, dv, dw, m, n, k);
11
12  left_ger_B_kernel<<<gridB, block2d>>>
13     (dB, dv, dw, beta, m, n, k);
14
15  cudaMemcpy(hB.data(), dB, bytesB, cudaMemcpyDeviceToHost);

```

效果分析

GPU 加速的优势主要来自上二对角化阶段的规则矩阵计算。Householder 中的 GEMV 和 GER 计算量大、数据并行度高，适合使用 GPU 大量线程并行处理。cuBLAS 版本通常比手写 kernel 更稳定，因为 cuBLAS 内部已经对访存合并、规约、线程块划分和不同矩阵规模做了充分优化；手写 kernel 虽然更贴合本实验的数据结构，也便于理解计算过程，但在调度和访存优化上很难完全超过成熟库函数。

不过，GPU 部分不能只看 kernel 时间。SVD 上二对角化过程中仍然存在 CPU 构造 Householder 向量、CPU-GPU 数据传输、kernel 启动和最终结果回传等开销。若矩阵规模较小，GPU 启动和数据搬移可能抵消计算收益；若矩阵规模较大，规则矩阵更新的计算量增加，GPU 的并行优势才更明显。因此，GPU 在本实验中的定位是加速上二对角化阶段，而 GKH 阶段仍更适合保留在 CPU 端，通过 Pthread 或 OpenMP 处理活动块并行。

四、 并行实现拓展与分析

前几次实验已经分别完成了 SIMD、Pthread、MPI 和 GPU/cuBLAS 版本的基本实现，本节不再重复已有实验结果。

(一) 并行方式拓展分析

OpenMP 的任务调度方式分析

在完成 GKH 活动块的 OpenMP 并行化后，我打算探究不同任务调度方式是否会对 GKH 迭代性能产生影响。分别采用 static、dynamic 和 guided 三种调度方式，并通过记录 GKH 迭代时间比较它们的实际效果。

static 调度——将活动块均匀分配给各线程

```
1 #pragma omp parallel for schedule(static)
2 for (int i = 0; i < static_cast<int>(blocks.size()); ++i)
3 {
4     if (blocks[i].r > blocks[i].l)
5     {
6         one_block_step(U, B, V,
7                         blocks[i].l, blocks[i].r);
8     }
9 }
```

静态调度在进入并行区域时便完成任务分配，运行过程中不再重新调度，因此具有较低的线程调度开销。当各活动块长度较为接近时，各线程承担的计算量相对均衡，static 调度能够获得较好的性能。但是，GKH 迭代产生的活动块长度并不完全一致。较长活动块需要执行更多次 Givens 旋转和 bulge chasing，即使各线程获得的活动块数量相同，实际计算量仍可能存在差异。

dynamic 调度——动态分配活动块，适合负载不均衡的情况

```
1 #pragma omp parallel for schedule(dynamic, 1)
```

动态调度以一个完整活动块为单位分配任务。线程处理完当前活动块后，可以继续领取尚未执行的活动块，因此能够缓解不同活动块长度造成的负载不均衡问题。该策略更适合 SVD 迭代后期形成多个不等长活动块的情况，但每次领取任务都需要 OpenMP 运行时参与调度。当活动块规模较小或活动块数量较少时，动态调度开销可能抵消负载均衡带来的收益。

guided 调度——开始分配较多任务，随后逐渐减小任务粒度

```
1 #pragma omp parallel for schedule(guided, 1)
```

guided 调度同样采用动态分配方式，但会根据剩余循环次数逐渐减小每次分配的任务数量。并行循环开始时采用较大的任务粒度，以减少调度次数；接近循环结束时采用较小的任务粒度，以改善尾部负载均衡。该方式试图在 static 调度的低开销和 dynamic 调度的负载均衡能力之间取得折中。

表 1: OpenMP 不同调度方式下的 GKH 活动块性能对比

调度方式	线程数	GKH 耗时/ms	相对性能	特点说明
static	4	31320.3	1.000	固定分配任务，调度开销较低
dynamic	4	24768.4	1.265	动态领取活动块，负载均衡能力较强
guided	4	20448.9	1.532	逐渐减小任务粒度，兼顾负载与开销

关键发现

• static 调度性能最低

- **原因：**static 调度在进入并行区域时便将活动块固定分配给各线程。虽然各线程获得的活动块数量相对均匀，但不同活动块的长度并不一致。较长活动块需要执行更多次 Givens 旋转和 bulge chasing，因此活动块数量相同并不代表实际计算量相同。

- **线程等待：**当某些线程提前完成较短活动块后，仍需等待负责较长活动块的线程结束，造成并行区域末尾的同步等待。
- **典型表现：**GKH 迭代耗时达到 31320.3 ms，是三种调度方式中耗时最长的一种。

- **dynamic 调度显著提升性能**

- **性能提升：**dynamic 调度的 GKH 迭代时间为 24768.4 ms，与 static 调度相比减少了

$$\frac{31320.3 - 24768.4}{31320.3} \times 100\% \approx 20.9\%.$$

- **动态分配：**采用 `schedule(dynamic,1)`，每次向线程分配一个完整的 GKH 活动块。线程完成当前活动块后，可以继续领取尚未处理的活动块，从而减少部分线程提前空闲的情况。
- **适用性：**GKH 迭代过程中活动块的长度会随矩阵收敛状态不断变化，dynamic 调度能够根据线程的实际完成速度重新分配任务，因此比固定分配方式具有更好的负载均衡能力。

- **guided 调度性能最优**

- **最快耗时：**guided 调度的 GKH 迭代时间为 20448.9 ms，与 static 调度相比减少了 34.7%，与 dynamic 调度相比进一步减少了

$$\frac{24768.4 - 20448.9}{24768.4} \times 100\% \approx 17.4\%.$$

- **自适应粒度：**guided 调度在并行循环开始时采用相对较大的任务粒度，以减少频繁分配任务产生的运行时开销；随着剩余任务数量逐渐减少，再缩小任务粒度，以改善迭代末尾的负载均衡。
- **综合优势：**该调度方式既避免了 static 调度中活动块固定分配造成的线程等待，又降低了 dynamic 调度频繁领取任务产生的调度开销，因此在本次实验中取得了最短的 GKH 迭代时间。

需要说明的是，我在早晨测试三种方式的数据得到的数据量都偏大且不稳定（例如 gkh 耗时 9 万多毫秒），在中午 12 点多时测试得到的是符合预期的结果，因此选择中午测试的结果作为上表的数据并加以分析。实验结果说明，GKH 迭代产生的活动块在长度和计算量上存在一定差异，固定均匀分配活动块容易造成线程负载不均衡。dynamic 调度能够通过动态领取任务减少线程空闲，而 guided 调度进一步平衡了任务分配开销与线程负载，在本次测试中获得了最好的性能。

(二) 并行组合拓展分析

在分别完成 CPU 和 GPU 版本的 SVD 实验后，我在想 GPU 是否适合处理所有规模的矩阵，以及能否根据 SVD 不同阶段的计算特征，在 CPU 和 GPU 之间进行更合理的任务分配。因此，我打算探究矩阵规模对 CPU 和 GPU 执行效率的影响，并寻找二者之间的性能交叉点。在此基础上设计策略：小规模矩阵使用 CPU 完成上二对角化，大规模矩阵使用 cuBLAS，上二对角化结束后再统一由 CPU 完成 GKH 迭代。通过设置不同阈值并比较上二对角化时间和完整 SVD 总时间，进一步判断哪一种 CPU-GPU 任务划分方式更加快速。

实现思路及方法

CPU 版本与 cuBLAS 版本分别实现为 `to_bidiagonal_cpu()` 和 `to_bidiagonal_gpu_cublas()`，再由统一接口根据矩阵规模选择计算路径。

Listing 1: SVD 中 CPU-GPU 阈值选择策略

```

1  #ifndef SVD_GPU_THRESHOLD
2  #define SVD_GPU_THRESHOLD 256
3  #endif
4
5  Matrix to_bidiagonal(const Matrix &A,
6                      Matrix &U,
7                      Matrix &V)
8  {
9      const int scale =
10         std::min(A.rows(), A.cols());
11
12     if (scale < SVD_GPU_THRESHOLD)
13     {
14         return to_bidiagonal_cpu(A, U, V);
15     }
16
17     try
18     {
19         return to_bidiagonal_gpu_cublas(A, U, V);
20     }
21     catch (const std::exception &e)
22     {
23         std::cerr << "[hybrid] GPU failed, "
24                   << "falling back to CPU: "
25                   << e.what() << '\n';

```



```
26     return to_bidiagonal_cpu(A, U, V);  
27 }  
28 }
```

实验要求 $m \geq n$, 因此 $\min(m, n) = n$ 。对于本实验使用的方阵, 该值就是矩阵阶数 N 。当 N 小于阈值时使用 CPU, 避免 CUDA 初始化、显存申请和数据传输等固定开销; 当 N 达到阈值后使用 cuBLAS, 提高大规模 GEMV 和 GER 的计算吞吐量。代码还提供 GPU 失败后的 CPU 回退路径, 以保证程序的鲁棒性。本次正式测试中没有发生 GPU 回退。

实验设计与结果

实验在 AMD Ryzen 9 7845HX 处理器和 NVIDIA GeForce RTX 4060 Laptop GPU 上完成, CUDA 版本为 12.6, 编译优化等级为 -O2。测试矩阵规模为

128, 192, 256, 384, 512, 768, 1000.

每个阈值版本在同一进程中依次处理上述 7 种矩阵, 并使用 20260408 至 20260412 共 5 个随机种子。表中时间为一次混合规模测试中 7 个矩阵时间之和, 再对 5 次实验取平均值; 相对误差取全部测试中的最大值。

预实验表明, 384 阶矩阵的 CPU 和 GPU 上二对角化时间分别约为 119.8 ms 和 299.8 ms, CPU 更快; 512 阶矩阵的 CPU 和 GPU 时间分别约为 455.3 ms 和 330.1 ms, GPU 开始取得优势。因此, CPU 与 GPU 的性能交叉点位于 384 至 512 之间。

表 2: CPU-GPU 组合阈值策略性能对比

阈值	总时间/ms	上二对角化/ms	GKH 迭代/ms	最大相对误差	分析说明
128	20046.1	2036.1	18010.0	4.50×10^{-13}	中小矩阵过早进入 GPU
256	19888.1	2036.8	17851.3	4.50×10^{-13}	部分中等矩阵仍不适合 GPU
512	19605.6	1691.7	17913.8	4.50×10^{-13}	与 CPU-GPU 交叉点最接近

关键发现

- **阈值 128 和 256 启用 GPU 过早。**

当阈值为 128 时, 大部分中小矩阵都会进入 GPU 路径; 阈值为 256 时, 256 阶和 384 阶矩阵仍由 GPU 处理。由于这些矩阵的计算量不足以覆盖 CUDA 初始化和数据传输开销, 两种策略的上二对角化时间分别为 2036.1 ms 和 2036.8 ms, 性能基本相同。

- **阈值 512 取得最佳上二对角化性能。**

阈值 512 使 384 阶及以下矩阵使用 CPU, 512 阶及以上矩阵使用 cuBLAS, 与预实验测得的性能交叉点一致。其上二对角化时间为 1691.7 ms, 相比阈值 128 和 256 分别减少约 16.91% 和 16.94%。

- **完整 SVD 收益受到 GKH 阶段限制。**

阈值 512 的完整 SVD 总时间为 19605.6 ms, 相比阈值 128 和 256 分别减少约 2.20% 和 1.42%。其 GKH 时间约占总时间的

$$\frac{17913.8}{19605.6} \times 100\% \approx 91.37\%.$$

由于阈值策略只优化上二对角化阶段，较高的 GKH 时间占比稀释了端到端加速效果。三种阈值下的 GKH 时间差异不足 1%，主要来自系统负载和计时波动，应该不能归因于阈值策略。

(三) MPI 进程池改进分析

在 lab4 的 MPI 实验中，程序已经实现了主从式任务池：rank 0 作为 master 维护活动块任务池，rank > 0 的进程作为 worker 领取活动块并执行 GKH 迭代。原始实现能够保证结果正确，但 profiling 结果显示，该进程池设计仍有较大的改进空间。我打算根据 lab4 的 profiling 结果，尝试探究任务分发方式和矩阵状态同步方式对 MPI 性能的影响。

原 profiling 结果主要暴露出两个问题。第一，任务负载分布不均衡。在 4 进程和 8 进程测试中，主要计算任务均集中在 rank 1，其余 worker 的 tasks、work 和 compute_time 基本为 0。进一步检查代码发现，当初始任务池中只有一个大活动块时，master 会将该任务分配给一个 worker，并立即向其余 worker 发送停止消息。即使大活动块随后分裂出新的子块，已经退出的 worker 也无法重新参与计算。

第二，通信和数据复制开销较大。原始版本每次任务下发前都会同步完整的 U 、 B 和 V 矩阵。性能开销不仅来自 GKH 计算，还来自任务下发、结果回传以及 master 合并 patch 所产生的大规模数据搬移。

对于上述问题，我们从持久 worker、任务顺序和输入 patch 三个方面改进进程池。首先，空闲 worker 不再因为任务池暂时为空而立即退出，而是在 MPI 接收操作中保持等待。活动块分裂产生新任务后，master 可以继续将任务分发给处于等待状态的 worker；只有任务池清空且所有活动任务结束后，master 才统一发送停止消息。

```
1 std::vector<int> idle_workers;  
2 for (int p = 1; p < size; ++p)  
3 {  
4     idle_workers.push_back(p);  
5 }  
6  
7 dispatch_available();  
8  
9 while (active_workers > 0)  
10 {  
11     MPI_Probe(MPI_ANY_SOURCE,  
12               TAG_RESULT_HEADER,  
13               MPI_COMM_WORLD,
```

```

14         &status);
15
16     const int source = status.MPI_SOURCE;
17     recv_result_patch(source, elapsed, steps,
18                       new_blocks, U, B, V);
19
20     active_workers--;
21     idle_workers.push_back(source);
22
23     for (const Block &blk : new_blocks)
24     {
25         if (blk.r > blk.l)
26         {
27             task_pool.push_back(blk);
28         }
29     }
30
31     dispatch_available();
32 }
33
34 for (int p = 1; p < size; ++p)
35 {
36     send_stop_round(p);
37 }

```

其次，根据活动块长度估算任务工作量。长度为 len 的活动块需要执行多次 Givens 旋转并更新 U 、 B 和 V ，因此使用 len^2 近似表示任务计算量。每次分发前优先取出工作量最大的活动块，使较重任务尽早进入计算阶段。

```

1 static long long block_work(const Block &blk)
2 {
3     const int len = blk.r - blk.l + 1;
4     return 1LL * len * len;
5 }
6
7 static Block pop_next_task(
8     std::vector<Block> &task_pool)
9 {
10     auto it = std::max_element(
11         task_pool.begin(),

```

```

12     task_pool.end() ,
13     [](const Block &a, const Block &b)
14     {
15         return block_work(a) < block_work(b);
16     });
17
18     Block blk = *it;
19     task_pool.erase(it);
20     return blk;
21 }

```

最后,将任务输入由完整矩阵同步改为活动块 patch 同步。对于由 deflation 分裂得到的独立活动块 $[l, r]$, worker 只需要 U 和 V 中对应的列块,以及 $B[l:r, l:r]$ 子块。因此, master 不再为每个任务发送完整的 U 、 B 和 V , 而是提取当前活动块所需的局部数据。

```

1 static void send_block_state_to_worker(
2     int dest ,
3     const Block &blk ,
4     const Matrix &U,
5     const Matrix &B,
6     const Matrix &V)
7 {
8     std::vector<double> u_patch =
9         extract_cols(U, blk.l, blk.r);
10    std::vector<double> b_patch =
11        extract_square_block(B, blk.l, blk.r);
12    std::vector<double> v_patch =
13        extract_cols(V, blk.l, blk.r);
14
15    MPI_Send(u_patch.data(), u_patch.size(),
16            MPI_DOUBLE, dest,
17            TAG_TASK_U_PATCH, MPI_COMM_WORLD);
18    MPI_Send(b_patch.data(), b_patch.size(),
19            MPI_DOUBLE, dest,
20            TAG_TASK_B_PATCH, MPI_COMM_WORLD);
21    MPI_Send(v_patch.data(), v_patch.size(),
22            MPI_DOUBLE, dest,
23            TAG_TASK_V_PATCH, MPI_COMM_WORLD);
24 }

```

worker 接收 patch 后, 将其写回本地矩阵的对应位置, 再执行活动块内部的 GKH 追赶过程。

```
1 static void recv_block_state_from_master(  
2     const Block &blk,  
3     Matrix &U,  
4     Matrix &B,  
5     Matrix &V)  
6 {  
7     const int len = blk.r - blk.l + 1;  
8  
9     std::vector<double> u_patch(U.rows() * len);  
10    std::vector<double> b_patch(len * len);  
11    std::vector<double> v_patch(V.rows() * len);  
12  
13    MPI_Recv(u_patch.data(), u_patch.size(),  
14             MPI_DOUBLE, 0,  
15             TAG_TASK_U_PATCH,  
16             MPI_COMM_WORLD,  
17             MPI_STATUS_IGNORE);  
18    MPI_Recv(b_patch.data(), b_patch.size(),  
19             MPI_DOUBLE, 0,  
20             TAG_TASK_B_PATCH,  
21             MPI_COMM_WORLD,  
22             MPI_STATUS_IGNORE);  
23    MPI_Recv(v_patch.data(), v_patch.size(),  
24             MPI_DOUBLE, 0,  
25             TAG_TASK_V_PATCH,  
26             MPI_COMM_WORLD,  
27             MPI_STATUS_IGNORE);  
28  
29    apply_cols(U, blk.l, blk.r, u_patch);  
30    apply_square_block(B, blk.l, blk.r,  
31                       b_patch);  
32    apply_cols(V, blk.l, blk.r, v_patch);  
33 }
```

实验使用相同的-02 优化等级和随机种子 20260408, 分别测试 4 进程和 8 进程。原进程池采用 worker 提前停止、完整状态下发和普通栈式任务分配; 改

进进程池采用持久 worker、活动块 patch 下发和最大活动块优先策略。以下数据均取 1000×1000 矩阵的 GKH 阶段。

表 3: MPI 进程池改进前后的任务负载与性能

版本	进程数	主要计算 rank	GKH 时间/ms	负载分布说明
原进程池	4	rank 1	146613	rank 1 承担全部任务
改进进程池	4	rank 3	119900	rank 3 承担 990 个任务, rank 2 承担 4 个小任务
原进程池	8	rank 1	162681	rank 1 承担全部任务
改进进程池	8	rank 7	113489	rank 7 承担 990 个任务, rank 6 承担 4 个小任务

表 4: MPI 进程池改进前后的通信与复制开销

版本	进程数	任务输入 /MB	结果 patch /MB	临时申请 /MB	通信时间 /s	复制时间 /s
原进程池	4	22750.854	10166.806	20333.611	31.180	11.306
改进进程池	4	10166.806	10166.806	20333.611	18.183	9.298
原进程池	8	22750.854	10166.806	20333.611	35.150	12.140
改进进程池	8	10166.806	10166.806	20333.611	19.228	8.069

关键发现

- **活动块 patch 显著减少了任务输入通信量。**

原进程池的完整状态下发量为 22750.854 MB, 改进后活动块输入 patch 为 10166.806 MB, 减少了

$$\frac{22750.854 - 10166.806}{22750.854} \times 100\% \approx 55.3\%.$$

任务结果回传量仍为 10166.806 MB, 说明输入通信已经得到改善, 但 worker 向 master 回传结果仍然是主要通信开销之一。

- **通信时间和 GKH 时间均明显下降。**

4 进程下, 通信时间由 31.180 s 下降到 18.183 s, 减少约 41.7%; GKH 时间由 146613 ms 下降到 119900 ms, 减少约 18.2%。8 进程下, 通信时间由

35.150s 下降到 19.228s, 减少约 45.3%; GKH 时间由 162681 ms 下降到 113489 ms, 减少约 30.2%。由于四组实验的任务总数和总工作量完全一致, 因此性能提升主要来自任务输入通信和数据复制开销的降低, 而不是计算量减少。

- **持久 worker 修复了提前退出问题, 但负载仍未实现均衡。**

原进程池的 994 个任务全部集中在单个 worker。改进后, 第二个 worker 能够在活动块分裂后继续领取 4 个小任务, 说明持久 worker 机制已经生效。然而, 主要 worker 仍承担 990 个任务和几乎全部工作量, 其他 worker 的计算量仍接近 0。

其原因是当前矩阵的 GKH 迭代大多表现为一个 1×1 收敛块加一个继续迭代的大活动块。 1×1 块不需要继续计算, 因此任务池中长期只有一个非平凡活动块。进程池可以分发已经产生的独立任务, 却无法突破 GKH 算法本身提供的活动块并行度。

- **增加进程数没有获得线性加速。**

原进程池从 4 进程增加到 8 进程后, GKH 时间由 146613 ms 增加到 162681 ms, 说明新增进程没有获得有效任务, 反而增加了 MPI 管理与同步开销。改进进程池从 4 进程增加到 8 进程后, GKH 时间由 119900 ms 下降到 113489 ms, 只进一步减少约 5.3%。因此, 增加 worker 数量不能解决活动块数量不足的问题。

五、 总结

(一) 并行方法对比与效果分析

本文围绕 SVD 的上二对角化和 GKH 迭代两个主要阶段, 分别采用 SIMD、多线程、MPI 和 GPU 进行并行优化, 并进一步研究了 OpenMP 调度、CPU-GPU 阈值选择和 MPI 进程池改进。不同并行方法的特点如表所示。

表 5: SVD 各类并行方法对比

并行方法	主要优势	局限性
SIMD	加速连续向量的乘加运算, 适合 Householder 变换中的向量更新	受数据布局、访存连续性和向量宽度限制
Pthread/OpenMP	能够并行处理相互独立的 GKH 活动块, 其中 OpenMP 编程和调度更方便	并行度随活动块数量变化, 活动块较少时加速受限
MPI	可在多进程环境下分发活动块任务, 适合进一步扩展计算规模	矩阵状态传输和结果复制开销较大, 且大活动块可能造成负载不均衡
GPU/cuBLAS	能够利用高吞吐量的 GEMV 和 GER 运算加速上二对角化	小规模矩阵容易受到数据传输和 GPU 启动开销影响
CPU-GPU 组合	根据矩阵规模选择执行设备, 兼顾小规模延迟与大规模吞吐量	最佳阈值与软硬件环境有关, 需要通过实验确定

实验表明, SVD 不同阶段的计算结构存在明显差异, 因此不能简单地使用同一种并行方法处理全部计算。上二对角化以规则的矩阵向量运算为主, 更适合 SIMD 和 GPU; GKH 迭代包含活动块划分、收敛判断和追赶过程, 更适合使用 CPU 多线程或 MPI 动态调度。

(二) 关键优化收获

调度策略决定 GKH 活动块的并行效率 GKH 迭代需要根据超对角元素的收敛情况不断划分活动块, 各活动块的规模和计算量并不均匀。OpenMP 调度实验中, guided 调度耗时为 20448.9 ms, 优于 dynamic 的 24768.4 ms 和 static 的 31320.3 ms。static 将活动块固定分配给各线程, 当部分小活动块提前收敛时, 容易出现线程空闲; guided 则在迭代前期采用较大任务块降低调度开销, 在后期逐渐缩小任务粒度, 提高剩余活动块的分配灵活性。

启示: 对于 GKH 这类并行度随收敛过程动态变化的算法, 线程数并不是决定性能的唯一因素。只有使调度粒度与活动块的变化规律相适应, 才能减少线程等待并提高实际并行效率。

异构协作突破单一设备的性能限制 SVD 的两个主要阶段具有不同的计算特征, 因此采用 CPU-GPU 组合策略:

- 小规模矩阵在 CPU 上完成 Householder 上二对角化，避免显存分配、数据传输和 cuBLAS 算子启动开销；
- 大规模矩阵在 GPU 上调用 cuBLAS 完成 GEMV 和 GER，将规则且计算密集的子矩阵更新交给 GPU；
- 上二对角化结束后，由 CPU 继续执行包含收敛判断、活动块划分和 bulge chasing 的 GKH 迭代。

实验表明，CPU 与 GPU 的性能交叉点位于矩阵规模 384 至 512 之间，阈值 512 取得了较好的整体性能。该策略使上二对角化耗时降低约 16.9%，但总时间仅降低约 2.2%，原因是优化后 GKH 迭代仍占完整 SVD 时间的 91% 以上。

效果：CPU-GPU 组合避免了小规模任务盲目使用 GPU，同时保留了 GPU 在大规模 GEMV、GER 运算中的吞吐优势。实验也说明，局部算子的显著加速不一定能够带来同等幅度的端到端加速，优化方向应由完整 SVD 各阶段的时间占比决定。

profiling 推动 MPI 进程池的针对性改进 原 MPI 进程池以活动块作为计算任务，但任务分发时仍传输较大的矩阵状态，使通信和结果复制成为 GKH 并行化的重要开销。根据 profiling 结果，改进版本采用局部 patch 传输，只发送活动块实际需要的数据，并让工作进程完成任务后继续申请新任务。改进后任务输入量由 22750.854 MB 降低至 10166.806 MB；4 进程下通信时间由 31.180 s 降低至 18.183 s，GKH 耗时由 146613 ms 降低至 119900 ms。

启示：MPI 优化不能只关注进程数量，还要使通信范围与活动块的计算范围一致。进程池改进有效减少了无关矩阵数据的传输，但 GKH 后期通常只剩少量未收敛活动块，单个大活动块仍无法同时分配给多个进程。因此，通信优化提高了进程池效率，而活动块数量和任务粒度仍决定着 MPI 版本的扩展上限。

(三) 各次实验收获

SIMD 在 SIMD 实验中，我使用 ARM NEON 指令对 Householder 变换和 Givens 旋转中的向量乘加、行列更新进行向量化。O1 优化下，上二对角化平均耗时由 15317.66 ms 降低至 5497.23 ms，加速比约为 2.79。perf 结果显示，SIMD 版本的 IPC 由 0.37 提高至 0.39，L1 Cache 缺失次数由 32.5 亿次降低至 29.7 亿次，但总指令数有所增加。其原因是矩阵采用行主序存储，列更新需要额外的标量加载和向量拼接。实验说明，SIMD 性能不仅取决于向量指令，还受到矩阵布局 and 访存连续性的影响。

多线程 在多线程实验中，我分别使用 Pthread 线程池和 OpenMP 并行处理 GKH 活动块。Pthread 能够显式控制任务领取和线程同步，但任务分发、互

斥锁和条件变量会产生额外开销；OpenMP 代码更简洁，并可在任务较少时回退到串行执行。O2 条件下，OpenMP 的平均 GKH 耗时约为 33623.28 ms，优于串行版本的 36095.32 ms 和 Pthread 版本的 42816.72 ms。进一步测试表明，guided 调度能够适应活动块规模的动态变化，将耗时由 static 的 31320.3 ms 降低至 20448.9 ms。但各版本都无法消除单个活动块内部 bulge chasing 的前后依赖，因此可并行活动块数量仍是多线程加速的主要限制。

MPI 在 MPI 实验中，我构建了主从式 GKH 进程池，由主进程划分活动块并合并结果，工作进程执行局部 bulge chasing。profiling 发现，初始版本的计算主要集中在 rank 1，同时 994 次完整矩阵同步产生了 22750.854 MB 输入数据，通信和复制开销较大。根据这一结果，我将任务输入改为活动块局部 patch，并使 worker 持续申请任务。改进后任务输入量减少约 55.3%，4 进程通信时间由 31.180 s 降低至 18.183 s，GKH 耗时由 146613 ms 降低至 119900 ms。不过，GKH 后期独立活动块数量较少，仍然限制了多进程扩展能力。

GPU 在 GPU 实验中，我分别使用 cuBLAS 和手写 CUDA kernel 加速上二对角化中的 GEMV 与 GER，并通过转置视图解决 Matrix 行主序与 cuBLAS 列主序接口的差异。CPU、cuBLAS 和手写 kernel 的平均耗时分别为 2474.81 ms、660.20 ms 和 1006.65 ms，说明成熟库在访存组织、归约和线程调度方面优于基础手写实现。阈值实验进一步发现，CPU 与 GPU 的性能交叉点位于矩阵规模 384 至 512 之间，说明 GPU 加速还需要结合任务规模和数据传输开销选择合适的卸载范围。

六、 心得体会

通过本学期并行程序设计课程的学习，我对任务分解、数据并行、线程同步、进程通信和异构计算有了更加深入的理解。在实验中，我以奇异值分解为主线，先后使用 SIMD、Pthread、OpenMP、MPI 和 CUDA 对 SVD 进行优化。实验使我认识到，不同阶段具有不同的计算特征：上二对角化中的 GEMV 和 GER 适合 SIMD 与 GPU，而包含活动块划分、收敛判断和 bulge chasing 的 GKH 迭代更适合 CPU 多线程或多进程调度。

实验过程也让我认识到 profiling 对并行优化的重要性。通过分析缓存缺失、线程负载、MPI 通信、数据复制和 GPU 算子耗时，让我能够从实验现象中定位真实瓶颈。

最后，我想感谢王刚老师在理论课中的认真讲授，为我理解不同并行模型和完成各次实验奠定了重要基础。同时感谢助教团队录制实验视频，制作实验手册的无私奉献，特别感谢助教邹博闻学长在 svd 主题报告检查和点评中给予的耐心指导！

七、 大模型使用情况

本报告撰写过程中使用了大模型工具进行辅助，主要用于文字润色、报告结构整理、LaTeX 格式调整以及部分代码逻辑检查。对于 SIMD、多线程、MPI 和 GPU 等实验内容，大模型帮助梳理不同并行方法与 SVD 上二对角化、GKH 迭代之间的关系，使算法实现和性能分析的表述更加连贯。

在代码分析方面，大模型主要用于检查 OpenMP 调度、MPI 进程池和 CPU-GPU 阈值选择等代码的实现思路，并对编译测试方法提出建议。相关修改均结合原有实验代码完成，并通过实际编译运行、误差检查和 profiling 输出验证，而不是直接采用未经检验的生成结果。

在数据分析方面，大模型辅助计算不同版本之间的耗时变化比例，并将实验现象整理为适合报告的文字。大模型不作为实验数据来源，也不用于虚构或替代实验结果。

八、 代码链接

GitHub 链接：<https://github.com/Yanguozhuang/parallel>